K.R. MANGALAM UNIVERSITY

Database Management Lab File

(ENCS205)

Bachelor of
Technology in
Computer Science and Engineering

B. Tech CSE (Core)
4 Semester

School of Engineering & Technology

Name – Vishal Kumar

Roll No – 2401010276

Submitted To:

Mansi Kajal

EXPERIMENT-3

Database Schema for a customer-sale scenario
Customer(Cust id : integer, cust_name: string)
Item(item_id: integer, item_name: string, price: integer)
Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)
For the above schema, perform the following-

1. Create the tables with the appropriate integrity constraints.

```
CREATE DATABASE customer_sale_db;
▷Run
USE customer_sale_db;
▷Run | Select
CREATE TABLE Customer (
  cust_id   INT PRIMARY KEY,
  cust_name VARCHAR(50) NOT NULL
);
▷Run | Select
CREATE TABLE Item (
  item_id   INT PRIMARY KEY,
  item_name VARCHAR(50) NOT NULL,
  price     INT NOT NULL CHECK (price > 0)
);
▷Run | Select
CREATE TABLE Sale (
  bill_no   INT,
  bill_date DATE NOT NULL,
  cust_id   INT,
  item_id   INT,
  qty_sold  INT NOT NULL CHECK (qty_sold > 0),

  PRIMARY KEY (bill_no, item_id),

  FOREIGN KEY (cust_id) REFERENCES Customer(cust_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE,

  FOREIGN KEY (item_id) REFERENCES Item(item_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);  60ms
```

2. Insert around 10 records in each of the tables.

```
INSERT INTO Customer VALUES
(1,  'Varun Chauhan'),
(2,  'Jony Gautam'),
(3,  'Bhawana Sharma'),
(4,  'Vanshika'),
(5,  'Priyanka'),
(6,  'Anjali'),
(7,  'Mrinal'),
(8,  'Aryan'),
(9,  'Riya Singh'),
(10, 'Karan Verma');
▷Run | Select
INSERT INTO Item VALUES
(101, 'Pen',         10),
(102, 'Notebook',    50),
(103, 'Bag',         500),
(104, 'Bottle',      150),
(105, 'Shoes',       1200),
(106, 'Watch',       2000),
(107, 'Calculator',  350),
(108, 'Mouse',       300),
(109, 'Keyboard',    600),
(110, 'Headphones',  800);
▷Run | Select
INSERT INTO Sale VALUES
(1, CURDATE(), 1, 101, 5),
(1, CURDATE(), 1, 102, 2),
(2, CURDATE(), 2, 103, 1),
(3, CURDATE(), 3, 104, 3),
(4, CURDATE(), 4, 105, 1),
(5, CURDATE(), 5, 106, 2),
(6, CURDATE() - INTERVAL 1 DAY, 6, 107, 1),
(7, CURDATE() - INTERVAL 2 DAY, 7, 108, 4),
(8, CURDATE() - INTERVAL 3 DAY, 8, 109, 2),
(9, CURDATE() - INTERVAL 4 DAY, 9, 110, 1);  AffectedRows: 10  27ms
```

3. List all the bills for the current date with the customer names and item numbers.

```
SELECT s.bill_no, s.bill_date, c.cust_name, s.item_id
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
WHERE s.bill_date = CURDATE();  7ms
```

bill_no int	bill_date date	cust_name varchar	item_id int
1	2026-03-01	Varun Chauhan	101
1	2026-03-01	Varun Chauhan	102
2	2026-03-01	Jony Gautam	103
3	2026-03-01	Bhawana Sharma	104
4	2026-03-01	Vanshika	105
5	2026-03-01	Priyanka	106

4. List the total Bill details with the quantity sold, price of the item and the final amount.

```
SELECT
  s.bill_no,
  i.item_name,
  s.qty_sold,
  i.price,
  (s.qty_sold * i.price) AS final_amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;  25ms
```

bill_no int	item_name varchar	qty_sold int	price int	final_amount bigint
1	Pen	5	10	50
1	Notebook	2	50	100
2	Bag	1	500	500
3	Bottle	3	150	450
4	Shoes	1	1200	1200
5	Watch	2	2000	4000
6	Calculator	1	350	350
7	Mouse	4	300	1200
8	Keyboard	2	600	1200
9	Headphones	1	800	800

5. List the details of the customer who have bought a product which has a price>200.

```
SELECT DISTINCT c.*
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
JOIN Item i ON s.item_id = i.item_id
WHERE i.price > 200;  6ms
```

cust_id int	cust_name varchar
2	Jony Gautam
4	Vanshika
5	Priyanka
6	Anjali
7	Mrinal
8	Aryan
9	Riya Singh

6. Give a count of how many products have been bought by each customer.

```
SELECT
  c.cust_name,
  SUM(s.qty_sold) AS total_products
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
GROUP BY c.cust_name;  15ms
```

cust_name varchar	total_products decimal
Varun Chauhan	7
Jony Gautam	1
Bhawana Sharma	3
Vanshika	1
Priyanka	2
Anjali	1

7. Give a list of products bought by a customer having cust_id as 5.

```
SELECT i.item_name, s.qty_sold
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.cust_id = 5;  6ms
```

item_name varchar	qty_sold int
Watch	2

8. List the item details which are sold as of today.

```
SELECT DISTINCT i.*
FROM Item i
JOIN Sale s ON i.item_id = s.item_id
WHERE s.bill_date = CURDATE();  6ms
```

item_id int	item_name varchar	price int
101	Pen	10
102	Notebook	50
103	Bag	500
104	Bottle	150
105	Shoes	1200
106	Watch	2000

]

9. Create a view which lists out the bill_no, bill_date, cust_id, item_id, price, qty_sold, amount

```
CREATE VIEW Bill_Details AS
SELECT
  s.bill_no,
  s.bill_date,
  s.cust_id,
  s.item_id,
  i.price,
  s.qty_sold,
  (i.price * s.qty_sold) AS amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;  30ms
```

item_id int	item_name varchar	price int
101	Pen	10
102	Notebook	50
103	Bag	500
104	Bottle	150
105	Shoes	1200
106	Watch	2000

```
SELECT * FROM Bill_Details;
```

bill_no int	bill_date date	cust_id int	item_id int	price int	qty_sold int	amount bigint
1	2026-03-01	1	101	10	5	50
1	2026-03-01	1	102	50	2	100
2	2026-03-01	2	103	500	1	500
3	2026-03-01	3	104	150	3	450
4	2026-03-01	4	105	1200	1	1200
5	2026-03-01	5	106	2000	2	4000
6	2026-02-28	6	107	350	1	350
7	2026-02-27	7	108	300	4	1200
8	2026-02-26	8	109	600	2	1200
9	2026-02-25	9	110	800	1	800

10. Create a view listing daily sales date-wise for the last one week

```
CREATE VIEW Weekly_Sales AS
SELECT
  bill_date,
  SUM(i.price * s.qty_sold) AS daily_total_sales
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE bill_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY bill_date
ORDER BY bill_date;  28ms
```

```
SELECT * FROM Weekly_Sales;  10ms
```

bill_date date	daily_total_sales decimal
2026-02-25	800
2026-02-26	1200
2026-02-27	1200
2026-02-28	350
2026-03-01	6300

```sql
USE customer_sale_db;
CREATE VIEW Bill_Details AS
SELECT
    s.bill_no,
    s.bill_date,
    s.cust_id,
    s.item_id,
    i.price,
    s.qty_sold,
    (i.price * s.qty_sold) AS amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;
```

```sql
SELECT * FROM Bill_Details;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

bill_no	bill_date	cust_id	item_id	price	qty_sold	amount
1	2026-02-28	1	101	10	5	50
1	2026-02-28	1	102	50	2	100
2	2026-02-28	2	103	500	1	500
3	2026-02-28	3	104	150	3	450
4	2026-02-28	4	105	1200	1	1200
5	2026-02-28	5	106	2000	2	4000
6	2026-02-27	6	107	350	1	350
7	2026-02-26	7	108	300	4	1200
8	2026-02-25	8	109	600	2	1200
9	2026-02-24	9	110	800	1	800

58	14:49:08	USE customer_sale_db		0 row(s) affected
59	14:49:08	CREATE VIEW Bill_Details AS SELECT s.bill_no, s.bill_date, s.cust_id, s.item_id, i.price, s.qty_...		0 row(s) affected
60	14:49:49	USE customer_sale_db		0 row(s) affected
61	14:49:49	SELECT * FROM Bill_Details LIMIT 0, 1000		10 row(s) returned

10. Create a view which lists the daily sales date wise for the last one week.

```
1  USE customer_sale_db;
2  CREATE VIEW Weekly_Sales AS
3  SELECT
4      bill_date,
5      SUM(i.price * s.qty_sold) AS daily_total_sales
6  FROM Sale s
7  JOIN Item i ON s.item_id = i.item_id
8  WHERE bill_date >= CURDATE() - INTERVAL 7 DAY
9  GROUP BY bill_date
10 ORDER BY bill_date;
11 SELECT * FROM Weekly_Sales;
```

Result Grid | Filter Rows:

bill_date	daily_total_sales
2026-02-24	800
2026-02-25	1200
2026-02-26	1200
2026-02-27	350
2026-02-28	6300

#	Time	Action	Message
62	14:52:04	USE customer_sale_db	0 row(s) affected
63	14:52:04	CREATE VIEW Weekly_Sales AS SELECT bill_date, SUM(i.price * s.qty_sold) AS daily_total_sales FRO...	0 row(s) affected
64	14:52:04	SELECT * FROM Weekly_Sales LIMIT 0, 1000	5 row(s) returned